

Data Warehouse Macrofinanciero  
Proyecto Guiado A→Z

Francisco Ramírez

11 de junio de 2026

# Índice general

|                                                             |          |
|-------------------------------------------------------------|----------|
| <b>1. Hito 1: Creación del Proyecto</b>                     | <b>2</b> |
| 1.1. Objetivo . . . . .                                     | 2        |
| 1.2. Resultado esperado . . . . .                           | 2        |
| 1.3. Paso 1: Crear la carpeta del proyecto . . . . .        | 3        |
| 1.4. Paso 2: Abrir el proyecto en VS Code . . . . .         | 3        |
| 1.5. Paso 3: Inicializar Git . . . . .                      | 3        |
| 1.6. Paso 4: Crear la estructura de carpetas . . . . .      | 4        |
| 1.7. Paso 5: Crear los archivos iniciales . . . . .         | 4        |
| 1.8. Paso 6: Configurar el archivo .gitignore . . . . .     | 5        |
| 1.9. Paso 7: Crear el README inicial . . . . .              | 5        |
| 1.10. Paso 8: Crear el entorno virtual . . . . .            | 6        |
| 1.11. Paso 9: Instalar dependencias mínimas . . . . .       | 6        |
| 1.12. Paso 10: Realizar el primer commit . . . . .          | 6        |
| 1.13. Checklist de validación . . . . .                     | 7        |
| 1.14. Entregable . . . . .                                  | 7        |
| 1.15. Próximo hito . . . . .                                | 8        |
| <b>2. Hito 2: Instalación y Configuración de PostgreSQL</b> | <b>9</b> |
| 2.1. Objetivo . . . . .                                     | 9        |
| 2.2. Resultado esperado . . . . .                           | 9        |
| 2.3. Paso 1: Verificar instalación de PostgreSQL . . . . .  | 10       |
| 2.4. Paso 2: Iniciar PostgreSQL . . . . .                   | 10       |
| 2.5. Paso 3: Conectarse al servidor . . . . .               | 10       |
| 2.6. Paso 4: Crear la base de datos . . . . .               | 11       |
| 2.7. Paso 5: Conectarse a la nueva base . . . . .           | 11       |
| 2.8. Paso 6: Crear archivo .env . . . . .                   | 11       |
| 2.9. Paso 7: Actualizar .env.example . . . . .              | 12       |
| 2.10. Paso 8: Crear módulo de conexión . . . . .            | 12       |
| 2.11. Paso 9: Probar la conexión . . . . .                  | 13       |

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| 2.12. Paso 10: Realizar una consulta desde Python . . . . .        | 13        |
| 2.13. Paso 11: Instalar extensiones de VS Code . . . . .           | 13        |
| 2.14. Paso 12: Crear primer script SQL . . . . .                   | 14        |
| 2.15. Checklist de validación . . . . .                            | 14        |
| 2.16. Entregable . . . . .                                         | 14        |
| 2.17. Próximo hito . . . . .                                       | 15        |
| <b>3. Hito 3: Diseño de las Fuentes de Datos e Ingesta Inicial</b> | <b>16</b> |
| 3.1. Objetivo . . . . .                                            | 16        |
| 3.2. ¿Por qué comenzamos con archivos CSV? . . . . .               | 16        |
| 3.3. Arquitectura de la ingesta . . . . .                          | 17        |
| 3.4. Fuentes utilizadas . . . . .                                  | 18        |
| 3.4.1. Fuente 1: Tipo de Cambio . . . . .                          | 18        |
| 3.4.2. Fuente 2: Balances Bancarios . . . . .                      | 18        |
| 3.5. Paso 1: Crear estructura de almacenamiento . . . . .          | 19        |
| 3.6. Paso 2: Crear archivo tipo_cambio.csv . . . . .               | 19        |
| 3.7. Paso 3: Crear archivo balances.csv . . . . .                  | 20        |
| 3.8. Paso 4: Crear script de exploración . . . . .                 | 20        |
| 3.9. Paso 5: Ejecutar exploración . . . . .                        | 21        |
| 3.10. Paso 6: Verificar estructura . . . . .                       | 21        |
| 3.11. Paso 7: Validaciones básicas . . . . .                       | 21        |
| 3.12. Paso 8: Crear capa processed . . . . .                       | 22        |
| 3.13. Paso 9: Guardar datos procesados . . . . .                   | 22        |
| 3.14. Paso 10: Documentar las fuentes . . . . .                    | 22        |
| 3.15. Entregables . . . . .                                        | 23        |
| 3.16. Checklist de validación . . . . .                            | 23        |
| 3.17. Próximo hito . . . . .                                       | 24        |
| <b>4. Hito 4: Diseño del Modelo Dimensional</b>                    | <b>25</b> |
| 4.1. Objetivo . . . . .                                            | 25        |
| 4.2. Resultado esperado . . . . .                                  | 25        |
| 4.3. Paso 1: Entender qué es un modelo dimensional . . . . .       | 26        |
| 4.4. Paso 2: Identificar los procesos de negocio . . . . .         | 26        |
| 4.5. Paso 3: Definir el grano . . . . .                            | 26        |
| 4.5.1. Grano de fact_tipo_cambio . . . . .                         | 27        |
| 4.5.2. Grano de fact_balance_mensual . . . . .                     | 27        |
| 4.6. Paso 4: Definir dimensiones . . . . .                         | 27        |
| 4.6.1. Dimensión tiempo . . . . .                                  | 27        |
| 4.6.2. Dimensión moneda . . . . .                                  | 28        |
| 4.6.3. Dimensión entidad financiera . . . . .                      | 28        |

|                                                                                      |           |
|--------------------------------------------------------------------------------------|-----------|
| 4.7. Paso 5: Definir tablas de hechos . . . . .                                      | 28        |
| 4.7.1. Hecho tipo de cambio . . . . .                                                | 28        |
| 4.7.2. Hecho balance mensual . . . . .                                               | 29        |
| 4.8. Paso 6: Definir claves . . . . .                                                | 29        |
| 4.8.1. Clave de negocio . . . . .                                                    | 29        |
| 4.8.2. Clave subrogada . . . . .                                                     | 30        |
| 4.9. Paso 7: Diseñar el esquema estrella . . . . .                                   | 30        |
| 4.10. Paso 8: Crear documento del modelo . . . . .                                   | 31        |
| 4.11. Paso 9: Crear tabla de definición del modelo . . . . .                         | 32        |
| 4.12. Paso 10: Definir indicadores derivados . . . . .                               | 32        |
| 4.13. Paso 11: Validar el diseño . . . . .                                           | 33        |
| 4.14. Checklist de validación . . . . .                                              | 33        |
| 4.15. Entregable . . . . .                                                           | 34        |
| 4.16. Próximo hito . . . . .                                                         | 34        |
| <b>5. Hito 5: Construcción de la Capa Staging (Bronze)</b>                           | <b>35</b> |
| 5.1. Objetivo . . . . .                                                              | 35        |
| 5.2. Resultado esperado . . . . .                                                    | 35        |
| 5.3. ¿Qué es la capa Staging? . . . . .                                              | 36        |
| 5.4. Arquitectura del Data Warehouse . . . . .                                       | 36        |
| 5.5. Paso 1: Crear carpeta para scripts SQL . . . . .                                | 37        |
| 5.6. Paso 2: Crear los schemas . . . . .                                             | 37        |
| 5.7. Paso 3: Ejecutar el script . . . . .                                            | 37        |
| 5.8. Paso 4: Crear script de tablas staging . . . . .                                | 38        |
| 5.9. Paso 5: Diseñar tabla tipo <i>cambio<sub>raw</sub></i> . . . . .                | 38        |
| 5.10. ¿Por qué usamos TEXT? . . . . .                                                | 38        |
| 5.11. Paso 6: Diseñar tabla <i>balance<sub>ancario<sub>raw</sub></sub></i> . . . . . | 39        |
| 5.12. Paso 7: Ejecutar el DDL . . . . .                                              | 40        |
| 5.13. Paso 8: Verificar tablas creadas . . . . .                                     | 40        |
| 5.14. Paso 9: Inspeccionar estructura . . . . .                                      | 40        |
| 5.15. Paso 10: Crear script de prueba . . . . .                                      | 40        |
| 5.16. Paso 11: Documentar la capa staging . . . . .                                  | 41        |
| 5.17. Paso 12: Crear consulta de monitoreo . . . . .                                 | 42        |
| 5.18. Buenas prácticas . . . . .                                                     | 42        |
| 5.19. Checklist de validación . . . . .                                              | 42        |
| 5.20. Entregable . . . . .                                                           | 43        |
| 5.21. Próximo hito . . . . .                                                         | 43        |

|                                                                                  |           |
|----------------------------------------------------------------------------------|-----------|
| <b>6. Hito 5: Construcción de la Capa Staging (Bronze)</b>                       | <b>44</b> |
| 6.1. Objetivo . . . . .                                                          | 44        |
| 6.2. Resultado esperado . . . . .                                                | 44        |
| 6.3. ¿Qué es la capa Staging? . . . . .                                          | 45        |
| 6.4. Arquitectura del Data Warehouse . . . . .                                   | 45        |
| 6.5. Paso 1: Crear carpeta para scripts SQL . . . . .                            | 46        |
| 6.6. Paso 2: Crear los schemas . . . . .                                         | 46        |
| 6.7. Paso 3: Ejecutar el script . . . . .                                        | 46        |
| 6.8. Paso 4: Crear script de tablas staging . . . . .                            | 47        |
| 6.9. Paso 5: Diseñar tabla tipo_cambio <sub>r</sub> aw . . . . .                 | 47        |
| 6.10. ¿Por qué usamos TEXT? . . . . .                                            | 47        |
| 6.11. Paso 6: Diseñar tabla balance <sub>ancario</sub> r <sub>aw</sub> . . . . . | 48        |
| 6.12. Paso 7: Ejecutar el DDL . . . . .                                          | 49        |
| 6.13. Paso 8: Verificar tablas creadas . . . . .                                 | 49        |
| 6.14. Paso 9: Inspeccionar estructura . . . . .                                  | 49        |
| 6.15. Paso 10: Crear script de prueba . . . . .                                  | 49        |
| 6.16. Paso 11: Documentar la capa staging . . . . .                              | 50        |
| 6.17. Paso 12: Crear consulta de monitoreo . . . . .                             | 51        |
| 6.18. Buenas prácticas . . . . .                                                 | 51        |
| 6.19. Checklist de validación . . . . .                                          | 51        |
| 6.20. Entregable . . . . .                                                       | 52        |
| 6.21. Próximo hito . . . . .                                                     | 52        |
| <b>7. Hito 6: Construcción del Primer Proceso ETL</b>                            | <b>53</b> |
| 7.1. Objetivo . . . . .                                                          | 53        |
| 7.2. Resultado esperado . . . . .                                                | 54        |
| 7.3. Arquitectura del proceso . . . . .                                          | 54        |
| 7.4. Paso 1: Verificar archivos fuente . . . . .                                 | 54        |
| 7.5. Paso 2: Crear módulo de conexión . . . . .                                  | 55        |
| 7.6. Paso 3: Crear script de carga . . . . .                                     | 55        |
| 7.7. Paso 4: Importar librerías . . . . .                                        | 55        |
| 7.8. Paso 5: Leer tipo de cambio . . . . .                                       | 56        |
| 7.9. Paso 6: Leer balances . . . . .                                             | 56        |
| 7.10. Paso 7: Agregar metadatos . . . . .                                        | 56        |
| 7.11. Paso 8: Limpiar staging antes de cargar . . . . .                          | 57        |
| 7.12. Paso 9: Ejecutar truncado desde Python . . . . .                           | 57        |
| 7.13. Paso 10: Cargar tipo de cambio . . . . .                                   | 57        |
| 7.14. Paso 11: Cargar balances . . . . .                                         | 58        |
| 7.15. Paso 12: Confirmar carga . . . . .                                         | 58        |
| 7.16. Versión completa del script . . . . .                                      | 58        |

|                                                     |    |
|-----------------------------------------------------|----|
| 7.17. Paso 13: Ejecutar ETL . . . . .               | 60 |
| 7.18. Paso 14: Verificar en PostgreSQL . . . . .    | 60 |
| 7.19. Paso 15: Inspeccionar datos . . . . .         | 60 |
| 7.20. Paso 16: Crear script de validación . . . . . | 61 |
| 7.21. Paso 17: Crear orquestador . . . . .          | 61 |
| 7.22. Buenas prácticas . . . . .                    | 62 |
| 7.23. Checklist de validación . . . . .             | 62 |
| 7.24. Entregable . . . . .                          | 62 |
| 7.25. Próximo hito . . . . .                        | 63 |

# Capítulo 1

## Hito 1: Creación del Proyecto

### 1.1. Objetivo

El objetivo de este hito es construir la estructura inicial del proyecto y preparar el entorno de desarrollo que utilizaremos durante todo el manual.

Al finalizar este hito el estudiante tendrá:

- Un repositorio Git inicializado.
- Una estructura de carpetas profesional.
- Un entorno virtual de Python.
- Dependencias básicas instaladas.
- El primer commit del proyecto.

### 1.2. Resultado esperado

Al finalizar este hito, la estructura del proyecto deberá lucir de la siguiente forma:

```
dwh_macrofinanciero_rd/  
  
+-- data/  
|   +-- raw/  
|   \-- processed/  
  
+-- sql/
```

```
+++ etl/  
+++ app/  
+++ docs/  
+++ tests/  
  
+++ README.md  
+++ requirements.txt  
+++ .gitignore
```

### 1.3. Paso 1: Crear la carpeta del proyecto

Abrir PowerShell y navegar al directorio donde se almacenará el proyecto.  
Por ejemplo:

```
cd C:\Users\t490\Documents\MacroPol
```

Crear la carpeta principal:

```
mkdir dwh_macrofinanciero_rd  
cd dwh_macrofinanciero_rd
```

Verificar la ubicación actual:

```
pwd
```

### 1.4. Paso 2: Abrir el proyecto en VS Code

Desde la terminal:

```
code .
```

VS Code deberá abrir la carpeta recién creada.

### 1.5. Paso 3: Inicializar Git

Abrir la terminal integrada de VS Code y ejecutar:

```
git init
```

Salida esperada:

```
Initialized empty Git repository
```

Verificar el estado:

```
git status
```



## 1.6. Paso 4: Crear la estructura de carpetas

Crear las carpetas principales:

```
mkdir data
mkdir sql
mkdir etl
mkdir app
mkdir docs
mkdir tests
```

Crear las subcarpetas para almacenamiento de datos:

```
mkdir data\raw
mkdir data\processed
```

## 1.7. Paso 5: Crear los archivos iniciales

Crear los siguientes archivos:

- README.md
- requirements.txt
- .gitignore
- .env.example

La estructura del proyecto deberá verse así:

```
dwh_macrofinanciero_rd/
|
+-- data/
+-- sql/
+-- etl/
+-- app/
+-- docs/
+-- tests/
|
+-- README.md
+-- requirements.txt
+-- .gitignore
+-- .env.example
```

## 1.8. Paso 6: Configurar el archivo .gitignore

Crear el archivo:

```
.gitignore
```

con el siguiente contenido:

```
.venv/  
.env  
  
**pycache**/  
  
*.pyc  
*.pyo  
  
.vscode/  
  
data/processed/  
  
.DS_Store  
Thumbs.db
```

## 1.9. Paso 7: Crear el README inicial

Archivo:

```
README.md
```

Contenido sugerido:

```
# Data Warehouse Macrofinanciero RD  
  
Proyecto practico para construir una plataforma  
de inteligencia macrofinanciera utilizando:  
  
* PostgreSQL  
* Python  
* Streamlit  
* GitHub
```

Autor: Francisco Ramirez

### 1.10. Paso 8: Crear el entorno virtual

Crear el entorno virtual:

```
python -m venv .venv
```

Activarlo:

Windows:

```
.venv\Scripts\activate
```

Linux/Mac:

```
source .venv/bin/activate
```

La terminal deberá mostrar:

```
(.venv)
```

### 1.11. Paso 9: Instalar dependencias mínimas

Instalar los paquetes necesarios:

```
pip install pandas  
pip install sqlalchemy  
pip install psycopg2-binary  
pip install python-dotenv
```

Guardar las dependencias instaladas:

```
pip freeze > requirements.txt
```

### 1.12. Paso 10: Realizar el primer commit

Agregar todos los archivos al repositorio:

```
git add .
```

Verificar:

```
git status
```

Crear el primer commit:

```
git commit -m "Initial_project_structure"
```

Consultar el historial:

```
git log --oneline
```

Salida esperada:

```
3f2a9ab Initial project structure
```

### 1.13. Checklist de validación

Antes de continuar al siguiente hito, verificar:

- ☐ VS Code instalado y funcionando.
- ☐ Proyecto creado.
- ☐ Git inicializado.
- ☐ Estructura de carpetas creada.
- ☐ Entorno virtual activo.
- ☐ Dependencias instaladas.
- ☐ Primer commit realizado.

### 1.14. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Un proyecto abierto en VS Code.
- Un repositorio Git funcional.
- Un entorno virtual configurado.
- Una estructura base lista para desarrollar el Data Warehouse.

### **1.15. Próximo hito**

En el Hito 2 construiremos la infraestructura de base de datos:

- Instalación de PostgreSQL.
- Creación de la base de datos.
- Configuración de variables de entorno.
- Primera conexión Python → PostgreSQL.

## Capítulo 2

# Hito 2: Instalación y Configuración de PostgreSQL

### 2.1. Objetivo

El objetivo de este hito es preparar el motor de base de datos que servirá como núcleo del Data Warehouse macrofinanciero.

Al finalizar este hito el estudiante será capaz de:

- Instalar PostgreSQL.
- Crear una base de datos para el proyecto.
- Conectarse desde VS Code.
- Configurar variables de entorno.
- Conectarse desde Python.
- Ejecutar consultas SQL básicas.

### 2.2. Resultado esperado

Al finalizar este hito deberá existir:

- Una instancia funcional de PostgreSQL.
- Una base de datos llamada `macrofinanzas`.
- Un usuario con permisos adecuados.

- Un archivo `.env` para gestionar credenciales.
- Un script Python capaz de conectarse a la base de datos.

### 2.3. Paso 1: Verificar instalación de PostgreSQL

Abrir PowerShell y ejecutar:

```
psql --version
```

Salida esperada:

```
psql (PostgreSQL) 17.x
```

Si el comando no funciona:

- Verificar que PostgreSQL está instalado.
- Verificar que el directorio `bin` está en el `PATH`.

### 2.4. Paso 2: Iniciar PostgreSQL

Verificar que el servicio está activo.

En Windows:

1. Abrir Servicios.
2. Buscar PostgreSQL.
3. Verificar estado Running".

Alternativamente:

```
net start postgresql-x64-17
```

### 2.5. Paso 3: Conectarse al servidor

Abrir terminal:

```
psql -U postgres
```

Introducir la contraseña configurada durante la instalación.

Debería aparecer:

```
postgres=#
```

## 2.6. Paso 4: Crear la base de datos

Ejecutar:

```
CREATE DATABASE macrofinanzas ;
```

Verificar:

```
\l
```

La nueva base deberá aparecer en la lista.

Salir:

```
\q
```

## 2.7. Paso 5: Conectarse a la nueva base

```
psql -U postgres -d macrofinanzas
```

Verificar:

```
SELECT current_database();
```

Resultado esperado:

```
macrofinanzas
```

## 2.8. Paso 6: Crear archivo .env

En la raíz del proyecto crear:

```
.env
```

Contenido:

```
DB_HOST=localhost  
DB_PORT=5432  
DB_NAME=macrofinanzas  
DB_USER=postgres  
DB_PASSWORD=tu_password
```

**Importante:**

El archivo `.env` nunca debe enviarse a GitHub.



## 2.9. Paso 7: Actualizar .env.example

Crear una versión pública:

`.env.example`

Contenido:

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=macrofinanzas
DB_USER=postgres
DB_PASSWORD=your_password
```

Este archivo sí puede subirse al repositorio.

## 2.10. Paso 8: Crear módulo de conexión

Crear:

`etl/db.py`

```
from sqlalchemy import create_engine
from dotenv import load_dotenv
import os

load_dotenv()

HOST = os.getenv("DB_HOST")
PORT = os.getenv("DB_PORT")
DB = os.getenv("DB_NAME")
USER = os.getenv("DB_USER")
PASSWORD = os.getenv("DB_PASSWORD")

connection_string = (
    f"postgresql+psycopg2://{USER}:{PASSWORD}@{HOST}:{PORT}/{DB}"
)

engine = create_engine(connection_string)

print("Conexion_creada_correctamente.")
```

### 2.11. Paso 9: Probar la conexión

Ejecutar:

```
python etl/db.py
```

Salida esperada:

Conexión creada correctamente.

### 2.12. Paso 10: Realizar una consulta desde Python

Modificar temporalmente:

```
from sqlalchemy import text

with engine.connect() as conn:
    resultado = conn.execute(
        text("SELECT version();")
    )

    '''
    for fila in resultado:
        print(fila)
    '''
```

Ejecutar nuevamente:

```
python etl/db.py
```

Resultado esperado:

PostgreSQL 17.x ...

### 2.13. Paso 11: Instalar extensiones de VS Code

Instalar:

- PostgreSQL
- SQLTools
- SQLTools PostgreSQL Driver

Estas extensiones facilitarán la ejecución de consultas y exploración de objetos.

## 2.14. Paso 12: Crear primer script SQL

Crear:

sql/test\_connection.sql

Contenido:

```
SELECT current_database();
```

```
SELECT current_user;
```

```
SELECT version();
```

Ejecutarlo desde VS Code o psql.

## 2.15. Checklist de validación

Antes de continuar verificar:

- ☐ PostgreSQL instalado.
- ☐ Servicio iniciado.
- ☐ Base de datos creada.
- ☐ Archivo .env configurado.
- ☐ Python conectado a PostgreSQL.
- ☐ Consulta SQL ejecutada correctamente.

## 2.16. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Base de datos **macrofinanzas**.
- Módulo de conexión Python.
- Variables de entorno configuradas.
- VS Code conectado a PostgreSQL.
- Primer script SQL ejecutado exitosamente.

## **2.17. Próximo hito**

En el Hito 3 construiremos la primera capa de datos:

- Diseño de las fuentes.
- Creación de datos simulados.
- Organización de carpetas **raw** y **processed**.
- Primera ingesta hacia PostgreSQL.

## Capítulo 3

# Hito 3: Diseño de las Fuentes de Datos e Ingesta Inicial

### 3.1. Objetivo

El objetivo de este hito es construir la primera capa del flujo de datos. Al finalizar este hito el estudiante será capaz de:

- Comprender las fuentes de información del proyecto.
- Diseñar datasets alineados con las necesidades de negocio.
- Organizar correctamente las carpetas de datos.
- Crear archivos CSV simulados.
- Leer datos desde Python.
- Realizar validaciones básicas antes de cargar a PostgreSQL.

### 3.2. ¿Por qué comenzamos con archivos CSV?

En proyectos reales los datos suelen provenir de:

- APIs.
- Bases de datos.
- Excel.

- CSV.
- Servicios web.
- Plataformas regulatorias.

Sin embargo, durante las primeras etapas del proyecto es recomendable trabajar con archivos CSV.

Esto permite:

- Diseñar el modelo de datos.
- Construir el ETL.
- Validar la arquitectura.
- Evitar complejidad innecesaria.

Una vez validado el proceso, sustituiremos los CSV por fuentes reales.

### 3.3. Arquitectura de la ingesta

El flujo de trabajo será:

```
Fuente CSV
|
V
data/raw
|
V
Python
|
V
Validación
|
V
data/processed
|
V
PostgreSQL
```

### 3.4. Fuentes utilizadas

El proyecto utilizará dos fuentes principales.

#### 3.4.1. Fuente 1: Tipo de Cambio

Representa información similar a la publicada por el Banco Central.

Grano:

Una observación por fecha y moneda.

Variables:

- fecha
- moneda
- tasa\_compra
- tasa\_venta

Ejemplo:

```
fecha,moneda,tasa_compra,tasa_venta
2024-01-01,USD,58.50,58.80
2024-01-01,EUR,63.10,63.60
```

#### 3.4.2. Fuente 2: Balances Bancarios

Representa información similar a la publicada por la Superintendencia de Bancos.

Grano:

Una observación por mes y entidad.

Variables:

- periodo
- entidad\_cod
- entidad\_nombre
- tipo\_entidad

- activos\_totales
- cartera\_creditos
- depositos
- patrimonio
- resultado\_neto
- depositos\_me
- cartera\_me

Ejemplo:

```
periodo,entidad_cod,activos_totales
202401,B001,120000000
```

### 3.5. Paso 1: Crear estructura de almacenamiento

Verificar que existe:

```
data/
```

```
+-- raw/
```

```
+-- processed/
```

raw contendrá los datos originales.

processed contendrá los datos validados.

### 3.6. Paso 2: Crear archivo tipo\_cambio.csv

Crear:

```
data/raw/tipo_cambio.csv
```

Contenido mínimo:

```
fecha,moneda,tasa_compra,tasa_venta
2024-01-01,USD,58.50,58.80
2024-01-02,USD,58.55,58.85
2024-01-03,USD,58.60,58.90
2024-01-01,EUR,63.10,63.60
2024-01-02,EUR,63.20,63.70
```



### 3.7. Paso 3: Crear archivo balances.csv

Crear:

data/raw/balances.csv

Contenido mínimo:

```
periodo,entidad_cod,entidad_nombre,  
tipo_entidad,activos_totales,  
cartera_creditos,depositos,  
patrimonio,resultado_neto,  
depositos_me,cartera_me
```

```
202401,B001,Banco Alfa,  
Banco Multiple,120000000,  
85000000,95000000,  
15000000,1200000,  
25000000,18000000
```

### 3.8. Paso 4: Crear script de exploración

Crear:

etl/explore\_sources.py

```
import pandas as pd  
  
fx = pd.read_csv(  
    "data/raw/tipo_cambio.csv"  
)  
  
balances = pd.read_csv(  
    "data/raw/balances.csv"  
)  
  
print("\nTipo_de_Cambio")  
print(fx.head())  
  
print("\nBalances")  
print(balances.head())
```

### 3.9. Paso 5: Ejecutar exploración

Ejecutar:

```
python etl/explore_sources.py
```

Resultado esperado:

- Visualización de las primeras filas.
- Confirmación de lectura correcta.

### 3.10. Paso 6: Verificar estructura

Agregar:

```
print (fx.info())
```

```
print (balances.info())
```

Analizar:

- Número de filas.
- Número de columnas.
- Tipos de datos.
- Valores faltantes.

### 3.11. Paso 7: Validaciones básicas

Agregar:

```
print (
fx.isnull().sum()
)
```

```
print (
balances.isnull().sum()
)
```

Verificar:

- Valores nulos.

- Fechas inválidas.
- Tasas negativas.

### 3.12. Paso 8: Crear capa processed

Convertir tipos de datos:

```
fx["fecha"] = pd.to_datetime(
fx["fecha"]
)

fx["tasa_compra"] = (
fx["tasa_compra"]
. astype(float)
)

fx["tasa_venta"] = (
fx["tasa_venta"]
. astype(float)
)
```

### 3.13. Paso 9: Guardar datos procesados

```
fx.to_csv(
"data/processed/tipo_cambio.csv",
index=False
)

balances.to_csv(
"data/processed/balances.csv",
index=False
)
```

### 3.14. Paso 10: Documentar las fuentes

Crear:

```
docs/data_dictionary.md
```

Documentar:

- Nombre de variable.
- Descripción.
- Tipo de dato.
- Fuente.
- Frecuencia.

### **3.15. Entregables**

Al finalizar este hito deberá existir:

- Archivos CSV en raw.
- Archivos validados en processed.
- Script de exploración.
- Diccionario de datos.
- Validaciones básicas implementadas.

### **3.16. Checklist de validación**

- ☐ Existe carpeta raw.
- ☐ Existe carpeta processed.
- ☐ tipo\_cambio.csv creado.
- ☐ balances.csv creado.
- ☐ Python puede leer ambos archivos.
- ☐ Validaciones ejecutadas.
- ☐ Archivos procesados generados.

### **3.17. Próximo hito**

En el Hito 4 diseñaremos el modelo dimensional.  
Construiremos:

- Dimensión Tiempo.
- Dimensión Moneda.
- Dimensión Entidad Financiera.
- Hecho Tipo de Cambio.
- Hecho Balance Bancario.
- Primer esquema estrella.

## Capítulo 4

# Hito 4: Diseño del Modelo Dimensional

### 4.1. Objetivo

El objetivo de este hito es diseñar el modelo dimensional del Data Warehouse macrofinanciero.

Al finalizar este hito el estudiante será capaz de:

- Identificar el grano de cada tabla de hechos.
- Diferenciar dimensiones y hechos.
- Diseñar un esquema estrella.
- Definir claves de negocio y claves subrogadas.
- Documentar el modelo antes de implementarlo en PostgreSQL.

### 4.2. Resultado esperado

Al finalizar este hito tendremos definido el modelo lógico del Data Warehouse.

El modelo estará compuesto por:

- `dim_tiempo`
- `dim_moneda`
- `dim_entidad_financiera`

- `fact_tipo_cambio`
- `fact_balance_mensual`

### 4.3. Paso 1: Entender qué es un modelo dimensional

Un modelo dimensional organiza los datos para facilitar el análisis. Está compuesto por dos tipos principales de tablas:

- **Dimensiones:** describen el contexto del análisis.
- **Hechos:** contienen medidas numéricas que queremos analizar.

Ejemplo:

- La fecha es una dimensión.
- La entidad financiera es una dimensión.
- La moneda es una dimensión.
- La tasa de cambio es una medida.
- Los activos bancarios son una medida.

### 4.4. Paso 2: Identificar los procesos de negocio

En este proyecto tendremos dos procesos de negocio principales:

1. Seguimiento del mercado cambiario.
2. Seguimiento de balances bancarios mensuales.

Cada proceso generará una tabla de hechos.

### 4.5. Paso 3: Definir el grano

El grano indica qué representa una fila de una tabla de hechos. Esta decisión es fundamental.

**4.5.1. Grano de fact\_tipo\_cambio**

Una fila por fecha y moneda.

Ejemplo:

```
2024-01-01 | USD
2024-01-01 | EUR
2024-01-02 | USD
```

**4.5.2. Grano de fact\_balance\_mensual**

Una fila por mes y entidad financiera.

Ejemplo:

```
2024-01-31 | B001
2024-01-31 | B002
2024-02-29 | B001
```

**4.6. Paso 4: Definir dimensiones****4.6.1. Dimensión tiempo**

La dimensión tiempo será compartida por ambos hechos.  
Esto la convierte en una dimensión conformada.

```
dim\'_tiempo
```

```
sk\_tiempo
fecha
anio
mes
anio\_mes
nombre\_mes
trimestre
es\_fin\_de_mes
```



#### 4.6.2. Dimensión moneda

Esta dimensión describe las monedas utilizadas en el mercado cambiario.

dim\\_moneda

sk\\_moneda  
moneda\\_cod  
moneda\\_nombre

#### 4.6.3. Dimensión entidad financiera

Esta dimensión describe las entidades financieras.

dim\\_entidad\\_financiera

sk\\_entidad  
entidad\\_cod  
entidad\\_nombre  
tipo\\_entidad  
vigente\\_desde  
vigente\\_hasta  
es\\_actual

La dimensión entidad financiera se diseñará para soportar cambios históricos mediante SCD Tipo 2.

### 4.7. Paso 5: Definir tablas de hechos

#### 4.7.1. Hecho tipo de cambio

fact\\_tipo\\_cambio

sk\\_tiempo  
sk\\_moneda  
tasa\_compra  
tasa\_venta  
tasa\_promedio  
spread

Medidas:

- tasa\_compra
- tasa\_venta
- tasa\_promedio
- spread

#### 4.7.2. Hecho balance mensual

fact\\_balance\\_mensual

sk\\_tiempo  
 sk\\_entidad  
 activos\\_totales  
 cartera\\_creditos  
 depositos  
 patrimonio  
 resultado\\_neto  
 depositos\\_me  
 cartera\\_me

Medidas:

- activos\_totales
- cartera\_creditos
- depositos
- patrimonio
- resultado\_netto
- depositos\_me
- cartera\_me

### 4.8. Paso 6: Definir claves

#### 4.8.1. Clave de negocio

La clave de negocio proviene de la fuente original.  
 Ejemplos:

- moneda\_cod: USD, EUR.
- entidad\_cod: B001, B002.
- fecha 2024-01-31.

#### 4.8.2. Clave subrogada

La clave subrogada es creada por el Data Warehouse.  
Ejemplos:

- sk\_tiempo
- sk\_moneda
- sk\_entidad

Las tablas de hechos no deben depender directamente de las claves de negocio.

Deben conectarse mediante claves subrogadas.

### 4.9. Paso 7: Diseñar el esquema estrella

El esquema estrella del proyecto será:

```

dim_tiempo
|
|
dim_moneda ---- fact_tipo_cambio

      dim_tiempo
        |
        |

dim_entidad_financiera ---- fact_balance_mensual

```

La dimensión `dim_tiempo` conecta ambos hechos y permite análisis integrados.

## 4.10. Paso 8: Crear documento del modelo

Crear el archivo:

docs/modelo\\_dimensional.md

Contenido sugerido:

```
# Modelo Dimensional
```

```
## Procesos de negocio
```

1. Mercado cambiario
2. Balances bancarios mensuales

```
## Tablas de hechos
```

```
### fact_tipo_cambio
```

Grano:

Una fila por fecha y moneda.

Medidas:

- \* tasa\_compra
- \* tasa\_venta
- \* tasa\_promedio
- \* spread

```
### fact_balance_mensual
```

Grano:

Una fila por mes y entidad financiera.

Medidas:

- \* activos\_totales
- \* cartera\_creditos
- \* depositos
- \* patrimonio

```

* resultado_netto
* depositos_me
* cartera_me

## Dimensiones

* dim_tiempo
* dim_moneda
* dim_entidad_financiera

```

#### 4.11. Paso 9: Crear tabla de definición del modelo

Agregar al documento una tabla como la siguiente:

| Tabla                  | Tipo      | Grano                           |
|------------------------|-----------|---------------------------------|
| dim_tiempo             | Dimensión | Una fila por día                |
| dim_moneda             | Dimensión | Una fila por moneda             |
| dim_entidad_financiera | Dimensión | Una fila por versión de entidad |
| fact_tipo_cambio       | Hecho     | Una fila por fecha y moneda     |
| fact_balance_mensual   | Hecho     | Una fila por mes y entidad      |

#### 4.12. Paso 10: Definir indicadores derivados

A partir de este modelo construiremos indicadores como:

- Tipo de cambio promedio.
- Spread cambiario.
- ROA.
- ROE.
- Dolarización de depósitos.
- Dolarización de cartera.
- Concentración bancaria.

Fórmulas principales:

```
tasa_promedio = (tasa_compra + tasa_venta) / 2

spread = tasa_venta - tasa_compra

ROA = resultado_netto / activos_totales

ROE = resultado_netto / patrimonio

dolarizacion_depositos = depositos_me / depositos

dolarizacion_cartera = cartera_me / cartera_creditos
```

### 4.13. Paso 11: Validar el diseño

Antes de crear las tablas en PostgreSQL, responder:

1. ¿Cada tabla de hechos tiene un grano claro?
2. ¿Cada medida pertenece al grano definido?
3. ¿Las dimensiones describen correctamente los hechos?
4. ¿La dimensión tiempo puede usarse en ambos hechos?
5. ¿La entidad financiera requiere historia?

### 4.14. Checklist de validación

Antes de continuar verificar:

- ☐ Procesos de negocio identificados.
- ☐ Grano de cada hecho definido.
- ☐ Dimensiones definidas.
- ☐ Hechos definidos.
- ☐ Claves de negocio identificadas.
- ☐ Claves subrogadas definidas.
- ☐ Documento `modelo_dimensional.md` creado.

### 4.15. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Diseño lógico del Data Warehouse.
- Documento del modelo dimensional.
- Definición de grano.
- Lista de dimensiones.
- Lista de hechos.
- Lista de indicadores derivados.

### 4.16. Próximo hito

En el Hito 5 implementaremos la primera capa física del Data Warehouse:

- Creación de schemas.
- Creación de tablas staging.
- Preparación de la capa bronze.
- Primer script SQL del modelo.

## Capítulo 5

# Hito 5: Construcción de la Capa Staging (Bronze)

### 5.1. Objetivo

El objetivo de este hito es construir la primera capa física del Data Warehouse.

La capa staging será el punto de entrada de todos los datos al sistema. Al finalizar este hito el estudiante será capaz de:

- Crear schemas en PostgreSQL.
- Diseñar tablas de aterrizaje.
- Comprender la función de la capa Bronze.
- Implementar una estrategia de carga desacoplada de las transformaciones.
- Preparar el sistema para recibir datos desde Python.

### 5.2. Resultado esperado

Al finalizar este hito deberá existir la siguiente estructura lógica:

PostgreSQL

```
+-- staging  
|
```



```
+-- core
|
+-- mart
```

Y dentro del schema staging:

```
staging.tipo_cambio_raw
```

```
staging.balance_bancario_raw
```

### 5.3. ¿Qué es la capa Staging?

La capa staging es el área donde aterrizan los datos provenientes de las fuentes externas.

Su objetivo es almacenar los datos en su forma más cercana al origen.

No se realizan:

- Cálculos.
- Transformaciones complejas.
- Generación de indicadores.
- Limpieza profunda.

La regla general es:

Cargar primero. Transformar después.

### 5.4. Arquitectura del Data Warehouse

La arquitectura que construiremos será:

```
Fuentes
|
V
Staging (Bronze)
|
V
Core (Silver)
|
V
Mart (Gold)
```

## 5.5. Paso 1: Crear carpeta para scripts SQL

Verificar:

```
sql/
```

Crear:

```
sql/01_create_schemas.sql
```

## 5.6. Paso 2: Crear los schemas

Agregar:

```
CREATE SCHEMA IF NOT EXISTS staging;
```

```
CREATE SCHEMA IF NOT EXISTS core;
```

```
CREATE SCHEMA IF NOT EXISTS mart;
```

Guardar.

## 5.7. Paso 3: Ejecutar el script

Desde psql:

```
\i sql/01_create_schemas.sql
```

Verificar:

```
SELECT schema_name
FROM information_schema.schemata
WHERE schema_name IN
(
  'staging',
  'core',
  'mart'
);
```

Resultado esperado:

```
staging
core
mart
```

## 5.8. Paso 4: Crear script de tablas staging

Crear:

sql/02\\_staging\\_ddl.sql

## 5.9. Paso 5: Diseñar tabla tipo\_cambio<sub>raw</sub>

Agregar:

```
CREATE TABLE IF NOT EXISTS
staging.tipo_cambio_raw
(
  fecha TEXT,

  '''
moneda TEXT,

tasa_compra TEXT,

tasa_venta TEXT,

archivo_origen TEXT,

cargado_en TIMESTAMP
DEFAULT CURRENT_TIMESTAMP
'''

);
```

## 5.10. ¿Por qué usamos TEXT?

Esta es una práctica común en Data Engineering.

Ventajas:

- La carga nunca falla por formato.
- Se preserva el dato original.
- La validación ocurre posteriormente.
- Se facilita el diagnóstico de errores.

### 5.11. Paso 6: Diseñar tabla `balance_bancario_raw`

Agregar:

```
CREATE TABLE IF NOT EXISTS
staging.balance_bancario_raw
(
periodo TEXT,

'''
entidad_cod TEXT,

entidad_nombre TEXT,

tipo_entidad TEXT,

activos_totales TEXT,

cartera_creditos TEXT,

depositos TEXT,

patrimonio TEXT,

resultado_neto TEXT,

depositos_me TEXT,

cartera_me TEXT,

archivo_origen TEXT,

cargado_en TIMESTAMP
DEFAULT CURRENT_TIMESTAMP
'''

);
```

### 5.12. Paso 7: Ejecutar el DDL

Ejecutar:

```
\i sql/02_staging_ddl.sql
```

### 5.13. Paso 8: Verificar tablas creadas

Consultar:

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema='staging';
```

Resultado esperado:

```
tipo_cambio_raw
```

```
balance_bancario_raw
```

### 5.14. Paso 9: Inspeccionar estructura

Consultar:

```
SELECT
column_name,
data_type
FROM information_schema.columns
WHERE table_schema='staging'
AND table_name='tipo_cambio_raw';
```

Verificar que todas las columnas fueron creadas correctamente.

### 5.15. Paso 10: Crear script de prueba

Crear:

```
sql/test_staging.sql
```

Contenido:

```
SELECT *  
FROM staging.tipo_cambio_raw;
```

```
SELECT *  
FROM staging.balance_bancario_raw;
```

Actualmente ambas tablas deben estar vacías.

## 5.16. Paso 11: Documentar la capa staging

Crear:

docs/staging\_design.md

Documentar:

- Nombre de tabla.
- Fuente original.
- Frecuencia.
- Número esperado de columnas.
- Responsable de carga.

Ejemplo:

Tabla:  
staging.tipo\_cambio\_raw

Fuente:  
Banco Central

Frecuencia:  
Diaria

Objetivo:  
Almacenar datos crudos del mercado cambiario.

### 5.17. Paso 12: Crear consulta de monitoreo

Crear:

sql/monitor\_staging.sql

Contenido:

```
SELECT  
COUNT(*) AS total_filas  
FROM staging.tipo_cambio_raw;
```

```
SELECT  
COUNT(*) AS total_filas  
FROM staging.balance_bancario_raw;
```

Esta consulta se utilizará para monitorear futuras cargas.

### 5.18. Buenas prácticas

Durante este proyecto seguiremos las siguientes reglas:

1. Nunca modificar datos manualmente en staging.
2. Mantener los datos lo más cercanos posible a la fuente.
3. No realizar cálculos en staging.
4. Todas las transformaciones ocurren en la capa core.
5. Conservar metadatos de carga.

### 5.19. Checklist de validación

Antes de continuar verificar:

- ☐ Schema staging creado.
- ☐ Schema core creado.
- ☐ Schema mart creado.

- ☐ Tabla `tipo_cambio_raw_creada`.
- ☐ Tabla `balance_bancario_raw_creada`.
- ☐ Scripts SQL almacenados en la carpeta `sql`.
- ☐ Consultas de validación ejecutadas.

## 5.20. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Arquitectura física inicial del Data Warehouse.
- Schemas creados.
- Tablas staging creadas.
- Scripts SQL documentados.
- Capa Bronze lista para recibir datos.

## 5.21. Próximo hito

En el Hito 6 construiremos el primer proceso ETL.  
Aprenderemos a:

- Leer archivos CSV desde Python.
- Conectarnos a PostgreSQL.
- Cargar información a staging.
- Automatizar la ingesta.
- Validar que los datos fueron cargados correctamente.



## Capítulo 6

# Hito 5: Construcción de la Capa Staging (Bronze)

### 6.1. Objetivo

El objetivo de este hito es construir la primera capa física del Data Warehouse.

La capa staging será el punto de entrada de todos los datos al sistema. Al finalizar este hito el estudiante será capaz de:

- Crear schemas en PostgreSQL.
- Diseñar tablas de aterrizaje.
- Comprender la función de la capa Bronze.
- Implementar una estrategia de carga desacoplada de las transformaciones.
- Preparar el sistema para recibir datos desde Python.

### 6.2. Resultado esperado

Al finalizar este hito deberá existir la siguiente estructura lógica:

PostgreSQL

```
+-- staging  
|
```

```
+-- core
|
+-- mart
```

Y dentro del schema staging:

```
staging.tipo_cambio_raw
```

```
staging.balance_bancario_raw
```

### 6.3. ¿Qué es la capa Staging?

La capa staging es el área donde aterrizan los datos provenientes de las fuentes externas.

Su objetivo es almacenar los datos en su forma más cercana al origen.

No se realizan:

- Cálculos.
- Transformaciones complejas.
- Generación de indicadores.
- Limpieza profunda.

La regla general es:

Cargar primero. Transformar después.

### 6.4. Arquitectura del Data Warehouse

La arquitectura que construiremos será:

```
Fuentes
|
V
Staging (Bronze)
|
V
Core (Silver)
|
V
Mart (Gold)
```

## 6.5. Paso 1: Crear carpeta para scripts SQL

Verificar:

```
sql/
```

Crear:

```
sql/01_create_schemas.sql
```

## 6.6. Paso 2: Crear los schemas

Agregar:

```
CREATE SCHEMA IF NOT EXISTS staging;
```

```
CREATE SCHEMA IF NOT EXISTS core;
```

```
CREATE SCHEMA IF NOT EXISTS mart;
```

Guardar.

## 6.7. Paso 3: Ejecutar el script

Desde psql:

```
\i sql/01_create_schemas.sql
```

Verificar:

```
SELECT schema_name
FROM information_schema.schemata
WHERE schema_name IN
(
  'staging ',
  'core ',
  'mart '
);
```

Resultado esperado:

```
staging
core
mart
```

## 6.8. Paso 4: Crear script de tablas staging

Crear:

sql/02\\_staging\\_ddl.sql

## 6.9. Paso 5: Diseñar tabla tipo\_cambio<sub>raw</sub>

Agregar:

```
CREATE TABLE IF NOT EXISTS
staging.tipo_cambio_raw
(
  fecha TEXT,

  '''
moneda TEXT,

tasa_compra TEXT,

tasa_venta TEXT,

archivo_origen TEXT,

cargado_en TIMESTAMP
DEFAULT CURRENT_TIMESTAMP
'''

);
```

## 6.10. ¿Por qué usamos TEXT?

Esta es una práctica común en Data Engineering.

Ventajas:

- La carga nunca falla por formato.
- Se preserva el dato original.
- La validación ocurre posteriormente.
- Se facilita el diagnóstico de errores.

### 6.11. Paso 6: Diseñar tabla `balance_bancario_raw`

Agregar:

```
CREATE TABLE IF NOT EXISTS
staging.balance_bancario_raw
(
periodo TEXT,

'''
entidad_cod TEXT,

entidad_nombre TEXT,

tipo_entidad TEXT,

activos_totales TEXT,

cartera_creditos TEXT,

depositos TEXT,

patrimonio TEXT,

resultado_neto TEXT,

depositos_me TEXT,

cartera_me TEXT,

archivo_origen TEXT,

cargado_en TIMESTAMP
DEFAULT CURRENT_TIMESTAMP
'''

);
```

## 6.12. Paso 7: Ejecutar el DDL

Ejecutar:

```
\i sql/02_staging_ddl.sql
```

## 6.13. Paso 8: Verificar tablas creadas

Consultar:

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema='staging';
```

Resultado esperado:

```
tipo_cambio_raw
```

```
balance_bancario_raw
```

## 6.14. Paso 9: Inspeccionar estructura

Consultar:

```
SELECT
column_name,
data_type
FROM information_schema.columns
WHERE table_schema='staging'
AND table_name='tipo_cambio_raw';
```

Verificar que todas las columnas fueron creadas correctamente.

## 6.15. Paso 10: Crear script de prueba

Crear:

```
sql/test_staging.sql
```

Contenido:

```
SELECT *  
FROM staging.tipo_cambio_raw;
```

```
SELECT *  
FROM staging.balance_bancario_raw;
```

Actualmente ambas tablas deben estar vacías.

## 6.16. Paso 11: Documentar la capa staging

Crear:

docs/staging\_design.md

Documentar:

- Nombre de tabla.
- Fuente original.
- Frecuencia.
- Número esperado de columnas.
- Responsable de carga.

Ejemplo:

Tabla:  
staging.tipo\_cambio\_raw

Fuente:  
Banco Central

Frecuencia:  
Diaria

Objetivo:  
Almacenar datos crudos del mercado cambiario.

## 6.17. Paso 12: Crear consulta de monitoreo

Crear:

sql/monitor\_staging.sql

Contenido:

```
SELECT  
COUNT(*) AS total_filas  
FROM staging.tipo_cambio_raw;
```

```
SELECT  
COUNT(*) AS total_filas  
FROM staging.balance_bancario_raw;
```

Esta consulta se utilizará para monitorear futuras cargas.

## 6.18. Buenas prácticas

Durante este proyecto seguiremos las siguientes reglas:

1. Nunca modificar datos manualmente en staging.
2. Mantener los datos lo más cercanos posible a la fuente.
3. No realizar cálculos en staging.
4. Todas las transformaciones ocurren en la capa core.
5. Conservar metadatos de carga.

## 6.19. Checklist de validación

Antes de continuar verificar:

- ☐ Schema staging creado.
- ☐ Schema core creado.
- ☐ Schema mart creado.



- ☐ Tabla `tipo_cambio_raw_creada`.
- ☐ Tabla `balance_bancario_raw_creada`.
- ☐ Scripts SQL almacenados en la carpeta `sql`.
- ☐ Consultas de validación ejecutadas.

## 6.20. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Arquitectura física inicial del Data Warehouse.
- Schemas creados.
- Tablas staging creadas.
- Scripts SQL documentados.
- Capa Bronze lista para recibir datos.

## 6.21. Próximo hito

En el Hito 6 construiremos el primer proceso ETL.  
Aprenderemos a:

- Leer archivos CSV desde Python.
- Conectarnos a PostgreSQL.
- Cargar información a staging.
- Automatizar la ingesta.
- Validar que los datos fueron cargados correctamente.

## Capítulo 7

# Hito 6: Construcción del Primer Proceso ETL

### 7.1. Objetivo

El objetivo de este hito es construir el primer flujo completo de carga de datos.

Por primera vez conectaremos:

```
CSV
|
V
Python
|
V
PostgreSQL
```

Al finalizar este hito el estudiante será capaz de:

- Leer archivos CSV mediante Python.
- Conectarse a PostgreSQL utilizando SQLAlchemy.
- Cargar información en la capa staging.
- Automatizar el proceso de carga.
- Validar que los datos fueron cargados correctamente.

## 7.2. Resultado esperado

Al finalizar este hito los datos contenidos en:

`data/raw/tipo_cambio.csv`

`data/raw/balances.csv`

deberán encontrarse cargados en:

`staging.tipo_cambio_raw`

`staging.balance_bancario_raw`

## 7.3. Arquitectura del proceso

El flujo que construiremos será:

```
CSV
|
V
Pandas
|
V
SQLAlchemy
|
V
PostgreSQL
```

## 7.4. Paso 1: Verificar archivos fuente

Confirmar la existencia de:

`data/raw/tipo_cambio.csv`

`data/raw/balances.csv`

Verificar que pueden abrirse correctamente.

## 7.5. Paso 2: Crear módulo de conexión

Verificar el archivo:

etl/db.py

Contenido:

```
from sqlalchemy import create_engine
from dotenv import load_dotenv

import os

load_dotenv()

HOST = os.getenv("DB_HOST")
PORT = os.getenv("DB_PORT")
DB = os.getenv("DB_NAME")
USER = os.getenv("DB_USER")
PASSWORD = os.getenv("DB_PASSWORD")

connection_string = (
    f"postgresql+psycopg2://{USER}:{PASSWORD}@{HOST}:{PORT}/{DB}"
)

engine = create_engine(
    connection_string
)
```

## 7.6. Paso 3: Crear script de carga

Crear:

etl/load\_staging.py

## 7.7. Paso 4: Importar librerías

Agregar:

```
import pandas as pd

from db import engine
```

## 7.8. Paso 5: Leer tipo de cambio

Agregar:

```
fx = pd.read_csv(
    "data/raw/tipo_cambio.csv"
)

print(
    f"Filas_tipo_cambio: {len(fx)}"
)
```

## 7.9. Paso 6: Leer balances

Agregar:

```
balances = pd.read_csv(
    "data/raw/balances.csv"
)

print(
    f"Filas_balances: {len(balances)}"
)
```

## 7.10. Paso 7: Agregar metadatos

Agregar:

```
fx["archivo_origen"] = (
    "tipo_cambio.csv"
)

balances["archivo_origen"] = (
    "balances.csv"
)
```

Estos metadatos permiten rastrear el origen de cada carga.

### 7.11. Paso 8: Limpiar staging antes de cargar

Crear:

sql/truncate\_staging.sql

Contenido:

```
TRUNCATE TABLE
staging.tipo_cambio_raw;

TRUNCATE TABLE
staging.balance_bancario_raw;
```

Esta estrategia garantiza idempotencia.

### 7.12. Paso 9: Ejecutar truncado desde Python

Agregar:

```
from sqlalchemy import text

with engine.begin() as conn:

    """
    conn.execute(
        text(
            """
            TRUNCATE TABLE
            staging.tipo_cambio_raw;

            TRUNCATE TABLE
            staging.balance_bancario_raw;
            """
        )
    )
    """
```

### 7.13. Paso 10: Cargar tipo de cambio

Agregar:

```
fx.to_sql(  
    "tipo_cambio_raw",  
    con=engine,  
    schema="staging",  
    if_exists="append",  
    index=False  
)
```

### 7.14. Paso 11: Cargar balances

Agregar:

```
balances.to_sql(  
    "balance_bancario_raw",  
    con=engine,  
    schema="staging",  
    if_exists="append",  
    index=False  
)
```

### 7.15. Paso 12: Confirmar carga

Agregar:

```
print(  
    "Carga_completada."  
)
```

### 7.16. Versión completa del script

El archivo completo debe verse así:

```
import pandas as pd  
  
from sqlalchemy import text  
  
from db import engine  
  
fx = pd.read_csv(  

```

## CAPÍTULO 7. HITO 6: CONSTRUCCIÓN DEL PRIMER PROCESO ETL63

```
"data/raw/tipo_cambio.csv"
)

balances = pd.read_csv(
    "data/raw/balances.csv"
)

fx["archivo_origen"] = (
    "tipo_cambio.csv"
)

balances["archivo_origen"] = (
    "balances.csv"
)

with engine.begin() as conn:

    '''
    conn.execute(
        text(
            """
            TRUNCATE TABLE
            staging.tipo_cambio_raw;

            TRUNCATE TABLE
            staging.balance_bancario_raw;
            """
        )
    )

    '''

    fx.to_sql(
        "tipo_cambio_raw",
        con=engine,
        schema="staging",
        if_exists="append",
        index=False
    )

    balances.to_sql(
```



```
"balance_bancario_raw",
con=engine,
schema="staging",
if_exists="append",
index=False
)

print("Carga_completada.")
```

### 7.17. Paso 13: Ejecutar ETL

Desde terminal:

```
python etl/load_staging.py
```

Salida esperada:

Carga completada.

### 7.18. Paso 14: Verificar en PostgreSQL

Consultar:

```
SELECT COUNT(*)
FROM staging.tipo_cambio_raw;
```

Consultar:

```
SELECT COUNT(*)
FROM staging.balance_bancario_raw;
```

Los conteos deben coincidir con los CSV.

### 7.19. Paso 15: Inspeccionar datos

Consultar:

```
SELECT *
FROM staging.tipo_cambio_raw
LIMIT 10;
```

Consultar:

```
SELECT *  
FROM staging.balance_bancario_raw  
LIMIT 10;
```

## 7.20. Paso 16: Crear script de validación

Crear:

sql/check\_load.sql

Contenido:

```
SELECT  
COUNT(*) AS total_fx  
FROM staging.tipo_cambio_raw;  
  
SELECT  
COUNT(*) AS total_balances  
FROM staging.balance_bancario_raw;
```

## 7.21. Paso 17: Crear orquestador

Crear:

etl/run\_pipeline.py

Contenido inicial:

```
import subprocess  
  
subprocess.run(  
[  
    "python",  
    "etl/load_staging.py"  
])
```

Este archivo será el punto de entrada de todo el pipeline.

## 7.22. Buenas prácticas

Durante este proyecto seguiremos las siguientes reglas:

1. El ETL debe ser reproducible.
2. La carga debe ser idempotente.
3. No modificar datos manualmente.
4. Validar cada carga.
5. Mantener trazabilidad mediante metadatos.

## 7.23. Checklist de validación

Antes de continuar verificar:

- ☐ Python puede leer ambos CSV.
- ☐ Python puede conectarse a PostgreSQL.
- ☐ Staging se limpia correctamente.
- ☐ Los datos son cargados.
- ☐ Los conteos coinciden.
- ☐ El script puede ejecutarse múltiples veces.

## 7.24. Entregable

Al finalizar este hito el estudiante deberá disponer de:

- Script de carga funcional.
- Datos cargados en staging.
- Validaciones básicas.
- Pipeline reproducible.
- Primer flujo ETL completo.

## **7.25. Próximo hito**

En el Hito 7 construiremos la capa Core.  
Crearemos:

- Dimensión Tiempo.
- Dimensión Moneda.
- Dimensión Entidad Financiera.
- Claves subrogadas.
- Primera transformación desde staging hacia el modelo dimensional.